Where to Go Next with C++

CSC 240 Data Structures (Lyon College Fall 2025)

Marcus Birkenkrahe

December 23, 2025

What This Course Was Really Preparing You For

This course was not about learning "more C++." It was about learning how to **build programs deliberately**.

- **Memory awareness** You learned that data has a concrete representation in memory and that incorrect access leads to undefined behavior. Example: Explaining why accessing 'a[10]' in a 10-element array is a bug even if the program appears to run.
- **Separating interface from implementation** You practiced keeping **user interaction** separate from **program logic**. Example: Handling input/output in 'main', while computation happens in functions that can be reused and tested independently.
- **Structured program planning** You learned to think in stages instead of writing code all at once. A useful pipeline: Data → Representation → Logic → Functions → Validation Example: Decide what data you need, choose how to store it, define the rules that operate on it, write functions for those rules, and verify correctness with test cases.
- **Indirection and controlled access** Pointers and references taught you to reason about who owns data and who is allowed to modify it. Example: Passing an array to a function and predicting exactly what will change.
- **Abstraction you can still explain** You used classes and STL containers only after understanding the underlying operations. Example:

Iterating over a ‘map‘ and explaining what each key–value pair represents and how lookup works.

- **Reading and debugging real code** You learned to trace execution, follow data flow, and debug logical errors rather than guessing.

What to Avoid (For Now)

- Advanced templates and metaprogramming that hide control flow and memory.
- Mixing input/output with program logic, which makes testing and reuse hard.
- Treating containers as black boxes. You should understand performance tradeoffs: ‘std::map‘ provides ordered keys and O(log n) lookup and insertion — far better than linear search, but slower than direct indexing.
- Clever code that you cannot explain line by line.

What This Sets You Up For Next

- Data structures and algorithms
- Systems programming and operating systems
- Larger, multi-file C++ programs
- Reading and maintaining other people’s code

Books to Read Next (By Direction)

- **Data Structures and Algorithms in C++** — Goodrich, Tamassia, Mount A direct continuation of this course: arrays, lists, trees, maps, with attention to abstraction and implementation.
- **Computer Systems: A Programmer’s Perspective** — Bryant & O’Hallaron Builds directly on pointers, memory layout, and data representation.

- **Operating Systems: Three Easy Pieces** — Arpaci-Dusseau & Arpaci-Dusseau Conceptual and grounded in real system behavior (mostly in C).
- **A Tour of C++** — Bjarne Stroustrup Concise overview of modern C++ and how its pieces fit together.
- **Effective Modern C++** — Scott Meyers Best read after you understand common C++ pitfalls.
- **The Practice of Programming** — Kernighan & Pike Focuses on clarity, structure, testing, and correctness.

Kattis Practice Problems (By Topic)

You find these at open.kattis.com/problems, a competitive programming site. You don't have to solve these in C++ either, you can use Python or 20 other languages.

Topic	Problem Name	Focus / Why It Matters
Arrays	bijele	Fixed-size arrays, indexing
Arrays	coldputer science	Counting values, traversal
Arrays	parking	Accumulation with arrays
Arrays	grading	Mapping indices to meaning
Vectors	reverse	Iteration over dynamic containers
Vectors	everywhere	Growing containers safely
Vectors	above average	Aggregation and traversal
Vectors	statistics	Passing vectors to functions
Pointers	encodedmessage	Index arithmetic, memory layout
Pointers	closing the loop	Careful loop control and tracing
Pointers	statistics	Array decay / indirect access
Pointers	oddities	Clean control flow, correctness
Maps	babelfish	Dictionary lookup (key $\rightarrow$ value)
Maps	conformity	Counting frequencies with keys
Maps	election	Tallying values with maps
Maps	help a phd candidate out	Key–value reasoning

How to Use This

- Start with the simplest structure you understand.
- Then revisit the problem with a better-matched container.
- Be able to explain why the second version is better.

Final Takeaway

You now have a way to think before coding: separate what the user sees from what the program does, design data first, build logic on top of it, encapsulate that logic in functions, and validate with tests.

That mindset matters more than any single C++ feature.